Private protocol

The private messages have to do with various validations. They check whether the market is still available or the wallet has the money. The actor sets a time limit for itself for checking both validations when opening. If it does not receive a confirmation within this time, it sends itself a `ValidationsTimedOut`. This is private because it belongs to the internal logic of the actor.

There are also `Fail` and `Close`, which are the two sides of the bet's final state. `Close` is the happy path: the final state when all the checks—and settlements, in case of a bet win—have been successfully completed. `Fail` is the unhappy ending that occurs when the checks or settlements are not completed successfully:

```
object Bet {
  ...

  private final case class MarketOddsAvailable(                    ❶
      available: Boolean,
      marketOdds: Option[Double])
      extends Command
  private final case class RequestWalletFunds(                     ❶
      response: Wallet.UpdatedResponse)
      extends Command
  private final case class ValidationsTimedOut(seconds: Int)
      extends Command                                              ❶
  private final case class Fail(reason: String) extends Command    ❶
  private final case class Close(reason: String) extends Command   ❶
  ...
}
```

❶ These private classes conform to the internal logic of the actor.

Before using the command handler to link commands and states together, let's look at the actor's different states.